L I N U X

RESTAURANT SYSTEMS PLATFORM

PROJECT SPECIFICATION & TECHNICAL DOCUMENTATION

# Menu & Record Management System

*A Linux-hosted, offline-tolerant platform unifying menu engineering, floor operations, statutory record-keeping and audit-grade reporting for multi-outlet food service.*

| | | | |
|---|---|---|---|
| DOCUMENT ID | LNX-SPEC-2024-001 | VERSION | 1.0 — Release Candidate |
| CLASSIFICATION | Internal / Confidential | STATUS | Approved for build |
| PREPARED BY | Linux Product Engineering | PAGES | A4 · Paginated master |

DEBIAN GNU/LINUX · POSTGRESQL · PAGED.JS MASTER COPY

# Document Control

REVISION HISTORY · APPROVALS · CONVENTIONS

This specification defines the functional scope, system architecture and data-governance contract for the Linux Restaurant Menu & Record Management System. It is the controlling reference for engineering, QA, operations and audit stakeholders throughout the build and acceptance phases.

### REVISION HISTORY

| Version | Date | Author | Description of change |
|---|---|---|---|
| 0.1 | 2024-02-06 | Product Engineering | Initial outline; module decomposition and scope boundary agreed. |
| 0.4 | 2024-02-21 | Architecture Group | Layered architecture, offline sync model and schema draft added. |
| 0.7 | 2024-03-11 | Finance Systems | Ledger, tax compliance and daily sales record sections expanded. |
| 0.9 | 2024-03-29 | Security & Audit | RBAC matrix, encryption posture and DR targets ratified. |
| 1.0 | 2024-04-12 | Product Engineering | Release-candidate baseline issued for build sign-off. |

### APPROVALS

| Role | Accountable party | Approval basis |
|---|---|---|
| Product Owner | Director of Operations | Scope, priorities and acceptance criteria |
| Technical Architect | Head of Platform Engineering | Architecture, stack and integration contracts |
| Finance Controller | Group Finance | Ledger integrity, tax records and reporting |
| Security Officer | Information Security | Access control, encryption and recovery |

## Conventions Used in This Document

| | |
|---|---|
| **MUST** | A mandatory requirement. Deviation requires a signed exception from the Technical Architect. |
| **SHOULD** | A strong recommendation; deviation requires documented justification at design review. |
| **RECORD** | Any immutable or append-only row written to the audit ledger. Records are never hard-deleted. |
| **OUTLET** | A single physical trading location with its own floor plan, tax profile and stock ledger. |
| **SHIFT** | A bounded trading window opened and closed by an accountable cashier or manager. |

> **READING ORDER**
>
> Chapters 1 and 2 are required reading for all stakeholders. Chapters 3 to 5 are operational reference for product and store teams. Chapters 6 and 7 are the contract for finance, audit and platform engineering.

# Contents

SEVEN CHAPTERS · PAGE REFERENCES GENERATED AT RENDER TIME


**1 Executive Summary & System Overview** 5

1.1 Purpose, Scope and Boundary 5

1.2 Project Objectives and Success Metrics 5

1.3 Stakeholder Benefits 6

1.4 Core Modules at a Glance 6

**2 System Architecture & Technology Stack** 8

2.1 Architectural Principles 8

2.2 Layered Architecture 8

2.3 Technology Stack 9

2.4 Offline-First POS Synchronisation 10

2.5 Data Model Overview 11

**3 Menu Engineering & Catalog Management** 12

3.1 Catalog Hierarchy 12

3.2 Pricing Rules Engine 12

3.3 Recipe Costing and Yield 13

3.4 Dietary and Allergen Tagging 13

3.5 Multi-Channel Publishing 14

**4 Floor & Order Management** 15

4.1 Floor Plan and Table Engine 15

4.2 Order Lifecycle and State Machine 15

4.3 Kitchen Display Integration 16

4.4 Billing, Split Payments and Settlement 16

**5 Record & Audit Ledger Management** 17

5.1 The Append-Only Ledger 17

5.2 Daily Sales Record and Z-Report 17

5.3 Inventory and Batch Traceability 18

5.4 Procurement and Supplier Records 18

5.5 Waste Logging and Tax Compliance 18

**6 Business Intelligence & Analytics** 20

6.1 Metric Definitions 20

6.2 Dashboard Catalogue 20

6.3 Food Cost Variance Analysis 21

6.4 Shift Audits and Exception Reporting 21

6.5 Data Export Formats 21

**7 Security, RBAC & Disaster Recovery 23**

7.1 Identity and Access Management 23

7.2 Role Permission Matrix 23

7.3 Encryption and Key Management 24

7.4 Backup, Recovery and Continuity 25

7.5 Compliance and Retention 25

CHAPTER ONE

# Executive Summary & System Overview

Linux is a single, Linux-hosted platform that replaces the usual patchwork of spreadsheets, paper dockets, till firmware and standalone stock books with one governed record of what a restaurant sells, what it consumes, and what it earns. It is designed for outlets where a dropped network connection must never stop service, and where a tax inspector or group auditor must be able to reconstruct any trading day months later.

## 1.1 Purpose, Scope and Boundary

The system serves two distinct but inseparable purposes. Operationally, it is a menu and point-of-sale platform: it publishes prices, takes orders from waiters and QR menus, routes them to the kitchen, and settles bills. Administratively, it is a record-management system: it maintains an append-only ledger of every sale, every ingredient movement, every purchase and every wastage event, and exposes that ledger through auditable reports.

SCOPE BOUNDARY

| In scope for v1.0 | Explicitly out of scope |
|---|---|
| Menu catalogue, modifiers, combos and scheduled pricing | Payroll, rostering and time-and-attendance |
| Floor plans, table states, order capture and KDS routing | Third-party delivery rider dispatch and tracking |
| Split billing, multi-tender settlement and till reconciliation | Full double-entry general ledger accounting |
| Inventory, recipe depletion, batch traceability and waste logs | Warehouse robotics or automated weigh-station hardware |
| Supplier purchase orders and goods-received notes | Supplier-side portals or EDI with distributors |
| Analytics dashboards, exports and statutory tax reports | Customer loyalty points and marketing automation |

## 1.2 Project Objectives and Success Metrics

Each objective below is bound to a measurable acceptance threshold. A phase is not signed off until the corresponding metric is demonstrated in the staging environment against representative trading data.

OBJECTIVES AND ACCEPTANCE THRESHOLDS

| ID | Objective | Success metric |
|---|---|---|
| OBJ-01 | Unify menu, order and stock data in one governed store | Zero duplicate price lists; a single price change propagates to all channels within 30 seconds |
| OBJ-02 | Keep service running through network loss | POS continues to accept orders for at least 8 hours offline; reconciliation completes with zero lost or duplicated orders |
| OBJ-03 | Produce audit-grade daily records automatically | Z-report generated at shift close with no manual re-keying; variance against tender count under 0.1% |
| OBJ-04 | Make recipe-level food cost visible daily | Theoretical versus actual consumption variance reported per item every trading day |
| OBJ-05 | Reduce menu-related service errors | Order-entry errors attributable to menu ambiguity reduced by 60% within two quarters |
| OBJ-06 | Satisfy statutory record retention | All required records retrievable within 5 minutes for any date in the retention window |

## 1.3 Stakeholder Benefits

The platform is deliberately shaped around six stakeholder groups. Each group receives a distinct primary benefit, but all of them read from the same underlying ledger — which is what removes the reconciliation arguments that plague multi-system estates.

STAKEHOLDER VALUE MAP

| Stakeholder | Primary benefit | Key enabling capability |
|---|---|---|
| Owner / General Manager | One trusted number for revenue, cost and margin | Consolidated analytics with outlet and period drill-down |
| Head Chef | Recipe and yield control without spreadsheet maintenance | Versioned recipe cards with automatic plate-cost recalculation |
| Floor Manager | Live visibility of tables, tickets and bottlenecks | Floor map with per-table state and KDS timing alerts |
| Cashier / Server | Faster, less error-prone order entry | Modifier-driven ordering, table transfer and split billing |
| Finance & Audit | Defensible records with an unbroken chain of custody | Append-only ledger, hash-chained entries and exportable reports |
| IT / Linux Administrator | A small, legible, self-hosted footprint | Containerised services, systemd units, documented recovery runbooks |

## 1.4 Core Modules at a Glance

Seven modules make up the platform. They are separately deployable services, but they share a single PostgreSQL cluster and a single identity provider, which keeps referential integrity intact across module boundaries.

### MODULE RESPONSIBILITY SUMMARY

| Module | Responsibility | Primary outputs |
|---|---|---|
| Menu & Catalog | Items, categories, modifiers, combos, price schedules | Published menu versions per channel |
| Floor & Orders | Table state, order capture, transfers, merges | Order records and kitchen tickets |
| Kitchen Display | Ticket routing, preparation timing, bump control | Prep-time events and ready signals |
| Ledger & Inventory | Sales ledger, stock movements, batches, waste | Append-only record stream |
| Procurement | Suppliers, purchase orders, goods received | PO and GRN records with batch links |
| Analytics | Aggregation, variance, dashboards, exports | Reports, KPI snapshots, CSV/XLSX/PDF |
| Identity & Audit | Authentication, roles, permissions, audit trail | Access tokens and immutable audit events |

**DESIGN INTENT**

Operational speed and record integrity are in tension. Linux resolves it by making writes to the operational tables fast and local, while the ledger layer records the corresponding immutable event asynchronously within the same transaction boundary — so a waiter never waits, but nothing is ever lost.

CHAPTER TWO

# System Architecture & Technology Stack

Linux is deployed on a standard Debian GNU/Linux host — on-premise in the outlet's back office, or on a single cloud instance for multi-outlet groups. There is no proprietary runtime, no vendor lock-in and no component that cannot be inspected with ordinary Linux tooling.

## 2.1 Architectural Principles

- **Local-first operation.** Every terminal holds a durable local queue. If the LAN drops, service continues and the queue drains on reconnection.
- **One source of truth.** A single PostgreSQL cluster owns all mutable operational state. Reporting reads replicas; it never writes back.
- **Append-only records.** Financial and inventory facts are written once, hash-chained, and never updated in place. Corrections are new compensating entries.
- **Boring infrastructure.** Nginx, PostgreSQL, Redis and systemd. Every layer must be operable by a competent Linux administrator without specialist training.
- **Idempotent integration.** All mutating API calls carry a client-generated idempotency key so a retried request can never double-post a sale.

## 2.2 Layered Architecture

The platform is organised into four layers with strictly one-directional dependencies. Clients never reach the data layer directly; every request passes the reverse proxy and an authenticated service boundary.

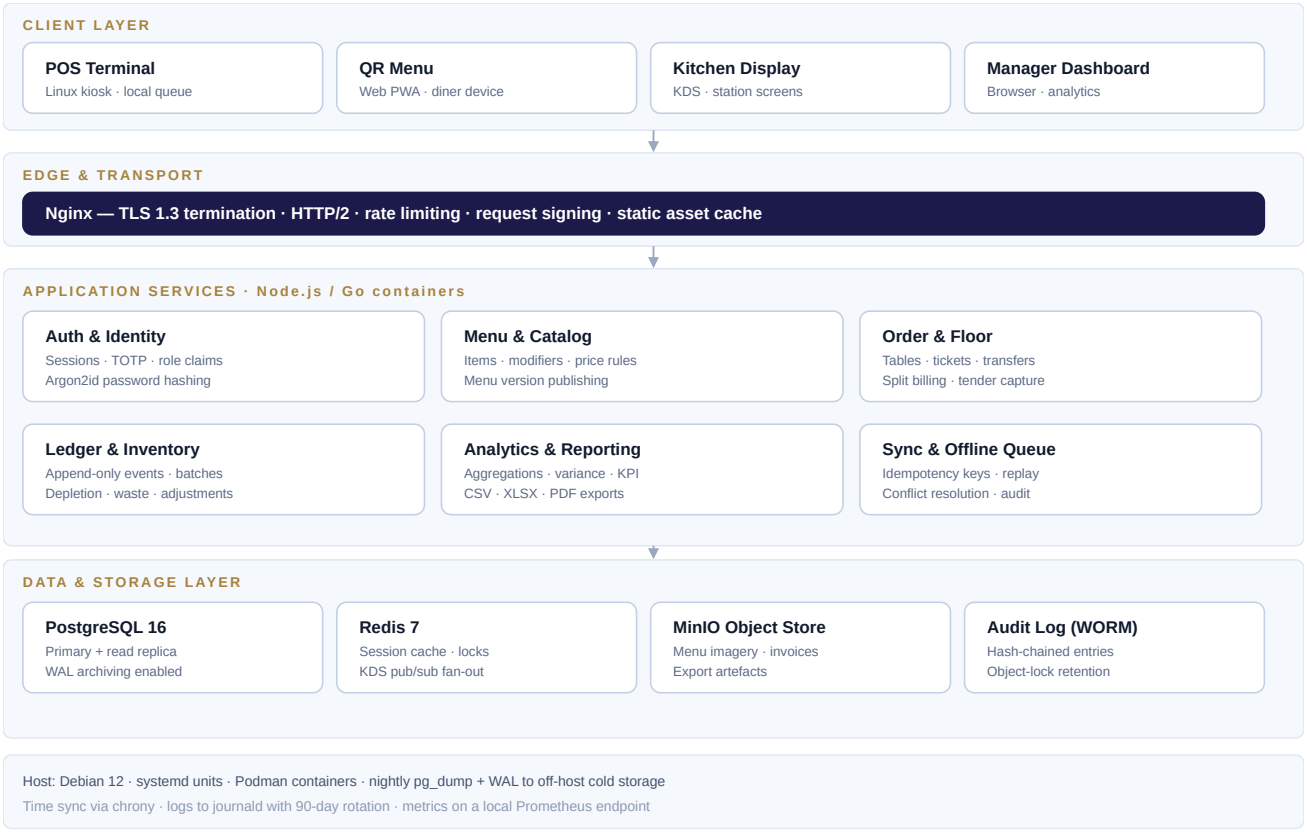



**FIGURE 2.1** — Layered architecture. Clients reach the service layer only through Nginx; the service layer owns all writes to the data layer.

## 2.3 Technology Stack

Every component below is open source, packaged for Debian, and replaceable without touching application logic. Version pinning is enforced by container image digests in the deployment manifest.

SELECTED STACK

| Layer | Component | Version | Selection rationale |
|---|---|---|---|
| Host OS | `Debian GNU/Linux` | `12` | Long-term support, minimal attack surface, ubiquitous administrator familiarity |
| Proxy | `Nginx` | `1.24` | TLS termination, static caching and request throttling in one binary |
| API runtime | `Node.js (LTS)` | `20.x` | Shared language across services; mature ecosystem for real-time sockets |
| Hot path | `Go` | `1.22` | Used for the sync and ledger writers where allocation predictability matters |
| Database | `PostgreSQL` | `16` | Strict transactions, JSONB flexibility, logical replication for read scaling |
| Cache / bus | `Redis` | `7.2` | Sub-millisecond pub/sub for KDS fan-out and distributed locks |
| Objects | `MinIO` | `RELEASE.2024` | S3-compatible, self-hosted, supports object-lock for WORM retention |
| Front end | `TypeScript + Vite` | `5.x` | Typed contracts shared with the API; small bundles for kitchen tablets |
| Containers | `Podman` | `4.9` | Daemonless, rootless operation — better default posture on a store server |

## 2.4 Offline-First POS Synchronisation

Loss of connectivity is treated as an expected operating condition rather than an error. Each POS terminal maintains a local SQLite queue in write-ahead mode. Orders, payments and table-state changes are committed locally first, then streamed to the server.

SYNCHRONISATION CONTRACT

| Step | Stage | Behaviour |
|---|---|---|
| 1 | Local commit | Terminal writes the transaction to SQLite and returns control to the operator immediately. |
| 2 | Queue & sign | The record is assigned a UUIDv7 idempotency key and signed with the terminal's device key. |
| 3 | Replay | On reconnection the queue is drained in causal order; the server rejects duplicates by key. |
| 4 | Reconcile | Terminal pulls a server-side delta and marks any locally divergent record for manager review. |

> **CONFLICT POLICY**
>
> Two terminals editing the same table concurrently is resolved by server-assigned logical sequence, not wall-clock time. The losing write is never discarded: it is retained as an exception record so the manager can see exactly what was attempted.

## 2.5 Data Model Overview

The schema is normalised to third normal form for operational tables, with JSONB reserved for genuinely variable payloads such as modifier selections and printer routing rules.

PRINCIPAL ENTITIES

| Entity | Key fields | Notes |
|---|---|---|
| menu_item | id, sku, name, category_id, base_price | Soft-retired, never deleted, so historical orders stay resolvable |
| item_variant | id, item_id, size, price_delta | Portion and size dimension |
| modifier_group | id, name, min_select, max_select | Drives both pricing and kitchen instructions |
| recipe_line | id, item_id, ingredient_id, qty, yield_pct | Basis for automatic depletion on sale |
| floor_table | id, zone, seats, state, party_id | State machine: free, seated, ordered, billed, dirty |
| order_header | id, table_id, shift_id, opened_at, status | One per sitting; splits derive from it |
| ledger_entry | id, entry_type, amount, ref, prev_hash, hash | Append-only, hash-chained, immutable |
| stock_batch | id, ingredient_id, lot_no, qty_on_hand, expiry | FEFO consumption; full traceability to supplier GRN |
| audit_event | id, actor_id, action, entity, before, after | Written for every privileged mutation |

CHAPTER THREE

# Menu Engineering & Catalog Management

The catalogue is the commercial heart of the system. It holds not just what a dish is called and what it costs the guest, but what it costs the kitchen — and it keeps that second number honest as supplier prices move.

## 3.1 Catalog Hierarchy

Items are organised in a four-level hierarchy. The hierarchy is not merely cosmetic: it drives the QR menu navigation, the KDS station routing, the sales-mix report grouping and the tax category applied at settlement.

- **Menu** — a named publication, such as *Dinner*, *Weekend Brunch* or *Bar Bites*. Each menu has its own validity window.
- **Category** — a section within the menu: *Starters*, *Mains*, *Desserts*, *Beverages*. Categories control display order and printer routing.
- **Item** — the sellable dish or drink, with a base price, tax class and prep station.
- **Variant & Modifier** — size, portion, and the option groups attached to the item (cooking preference, sides, add-ons).

An item belongs to exactly one category but may appear on several menus simultaneously. Retiring an item removes it from all active menus while leaving every historical order line intact and reprintable.

## 3.2 Pricing Rules Engine

Prices are rarely a single number. The engine evaluates rules in a fixed precedence order so that the outcome is always predictable and reproducible — a property that matters enormously during a tax audit.

RULE PRECEDENCE (LOWEST NUMBER EVALUATES FIRST)

| Order | Rule type | Example | Effect |
|---|---|---|---|
| 1 | Base price | Base price of the item master record | Starting value for all downstream rules |
| 2 | Variant delta | Large portion +2.50 | Additive adjustment per selected variant |
| 3 | Modifier delta | Extra cheese +1.20 | Summed across all selected paid modifiers |
| 4 | Time window | Happy hour 17:00–19:00, beverages −20% | Percentage applied to the running subtotal |
| 5 | Channel | Delivery aggregator uplift +12% | Applied only when the order source matches |
| 6 | Promotion | Two-course set menu at a fixed price | Overrides the running subtotal for the bundle |
| 7 | Rounding | Nearest 0.05 on cash tender | Terminal rounding applied at settlement only |

Every applied rule is written to the order line as a separate attribution row, so a receipt can always show the guest precisely why a price differed from the printed menu.

## 3.3 Recipe Costing and Yield

A recipe card links a sellable item to its ingredient lines. Each line declares a quantity, a unit and a yield percentage that accounts for trim and cooking loss. Plate cost is recomputed whenever an ingredient's weighted average cost changes.

**Plate cost** = Σ ( line quantity ÷ yield % × ingredient unit cost ) + packaging + allocated labour burden.

WORKED EXAMPLE — CHARRED OCTOPUS, 1 PORTION

| Ingredient | Qty | Unit | Yield | Unit cost | Line cost |
|---|---|---|---|---|---|
| Octopus, whole | 180 | g | 72% | 0.0210 / g | 5.25 |
| Smoked paprika | 2 | g | 100% | 0.0480 / g | 0.10 |
| Confit potato | 90 | g | 95% | 0.0062 / g | 0.59 |
| Extra virgin olive oil | 15 | ml | 100% | 0.0110 / ml | 0.17 |
| Lemon & herb garnish | 1 | portion | 100% | 0.1800 | 0.18 |
| Packaging allocation | 1 | — | — | — | 0.00 |
| **Total plate cost** | | | | | **6.29** |
| **Menu price** | | | | | **19.50** |
| **Food cost %** | | | | | **32.3%** |

> COSTING DISCIPLINE
>
> Ingredient costs use a moving weighted average recomputed at each goods-received note. A recipe card is versioned: changing a line creates a new version with an effective date, so a historical plate cost can always be reconstructed for any past trading day.

## 3.4 Dietary and Allergen Tagging

Tags are structured, not free text. Each tag carries a severity class that determines where it is surfaced — guest-facing menu, kitchen ticket, or both.

TAG CLASSES

| Class | Examples | Surfaced on |
|---|---|---|
| Regulated allergen | Gluten, crustaceans, eggs, fish, peanuts, soy, milk, nuts, celery, mustard, sesame, sulphites | Menu, KDS, receipt footer, audit export |
| Dietary claim | Vegetarian, vegan, halal, gluten-free | Menu and KDS |
| Kitchen instruction | Contains alcohol, served raw, high heat required | KDS only |
| Operational flag | Long prep, station-restricted, 86-risk | KDS and manager dashboard |

## 3.5 Multi-Channel Publishing

A single menu version can be published to several channels at once. Each channel receives a projection of the master catalogue rather than a copy, so a price correction in one place is never stale in another.

CHANNEL PROJECTIONS

| Channel | What is published | Refresh behaviour |
|---|---|---|
| QR diner menu | Item names, descriptions, prices, imagery, allergen icons | Edge-cached, revalidated within 30 seconds of publish |
| POS terminal | Full catalogue with modifiers, combos and printer routing | Pushed to terminals, acknowledged individually |
| Kitchen display | Item-to-station mapping, prep timers, modifier instructions | Live via Redis pub/sub |
| Delivery aggregators | Channel-uplifted pricing, availability, packaging SKUs | Scheduled push every 5 minutes with retry |
| Printed menu export | Typeset PDF with the current price list | On-demand generation, version-stamped |

Availability is a separate axis from pricing. An item can be marked *86* at any moment; the flag propagates to every channel within the same publish cycle and is removed only by an explicit manager action, never by an automatic timer.

CHAPTER FOUR

# Floor & Order Management

This chapter governs the minutes that matter most: from the moment a guest sits down to the moment the bill is settled and the table is released. Every state change here writes a record to the ledger described in Chapter 5.

## 4.1 Floor Plan and Table Engine

The floor plan is a set of zones, each containing tables at absolute coordinates on a scaled canvas. Tables carry a seat count, a shape and a state. Merging two tables produces a composite party without destroying either table's identity, which keeps historical occupancy reporting accurate.

TABLE STATE MACHINE

| State | Meaning | Permitted transitions |
|---|---|---|
| `free` | Available for seating | → seated |
| `seated` | Party assigned, no items ordered | → ordered, → free (walk-out) |
| `ordered` | At least one item sent to kitchen | → billed, → seated (void all) |
| `billed` | Bill presented, awaiting tender | → settled, → ordered (reopen) |
| `settled` | Payment captured, pending close | → dirty |
| `dirty` | Awaiting clearing and reset | → free |

A table cannot be released to free from settled without passing through dirty. This single constraint is what makes turnover-time reporting trustworthy, because the cleaning interval is always measured rather than assumed.

## 4.2 Order Lifecycle and State Machine

An order moves through a strictly monotonic sequence of states. Only two transitions are reversible, and both require a manager credential: reopening a settled bill and voiding a sent item.

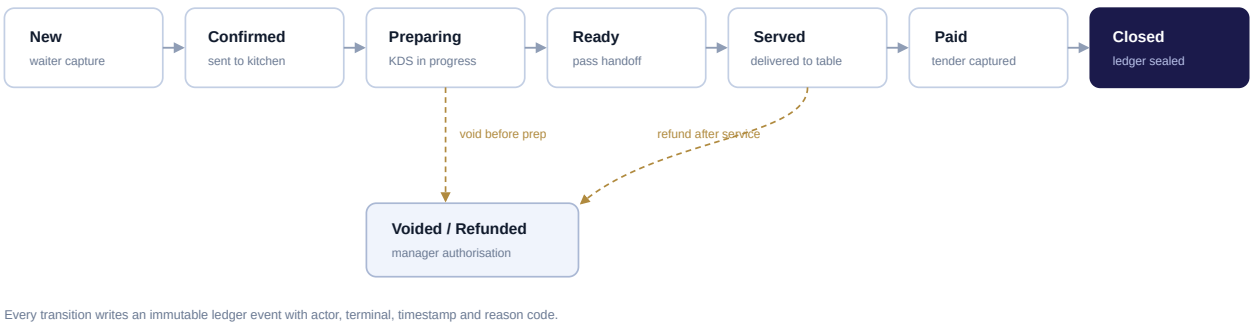



FIGURE 4.1 — Order lifecycle. Reversible transitions are manager-gated and always produce a compensating ledger entry.

## 4.3 Kitchen Display Integration

When an order is confirmed, the order service publishes a ticket event to Redis. Each KDS station subscribes to the item-to-station mapping and renders only the items routed to it, which avoids the common failure where a grill station is flooded with salad tickets.

- **Routing** — each item carries a primary and optional secondary station; combos split their components across stations automatically.
- **Timers** — a target prep time is attached per item; tickets past target turn amber at 100% and red at 140% of the target.
- **Bump control** — a ticket is bumped per station, and the guest-facing order only reaches Ready when all contributing stations have bumped.
- **Recall** — a bumped ticket can be recalled within 10 minutes without creating a new order line, but the recall is recorded as an event.
- **Degraded mode** — if the KDS loses connectivity it falls back to a printed expediter chit so service never stalls.

## 4.4 Billing, Split Payments and Settlement

Billing supports both item-level and amount-level splitting. A split never mutates the parent order: it produces child bill documents that reference the parent, so the original order remains a complete and auditable record.

SUPPORTED SETTLEMENT PATTERNS

| Pattern | Mechanism | Recorded as |
|---|---|---|
| Single tender | One payment covering the full balance | One settlement entry |
| Equal split by seats | Total divided by party size, rounding absorbed by the first share | N child bills with a rounding attribution row |
| Item split | Each guest is assigned specific order lines | N child bills referencing line IDs |
| Mixed tender | Cash plus card plus voucher against one bill | N settlement entries against one bill |
| Partial payment | Deposit or part-payment before service completes | Settlement entry with an open balance flag |
| House account | Charged to a named corporate account | Settlement entry with a receivable reference |

Card processing is delegated to the terminal's certified payment application; Linux never stores a primary account number. It records only the authorisation reference, the masked last four digits, the scheme and the amount — which is sufficient for reconciliation and compliant with card-industry data-minimisation rules.

> **TILL RECONCILIATION**
>
> At shift close the cashier declares counted tender by denomination. The system compares declared against expected, and any variance above the configured tolerance (default 0.1%) requires a written explanation before the shift can be sealed.

CHAPTER FIVE

# Record & Audit Ledger Management

This chapter is the reason the platform exists. Everything the restaurant does commercially or operationally resolves into a record — and every record is written once, chained to its predecessor, and retained for the statutory period.

## 5.1 The Append-Only Ledger

The ledger is a single ordered table. Rows are inserted, never updated and never deleted. Each row stores the hash of its predecessor, producing a chain that makes silent tampering detectable: altering any historical row invalidates every hash after it.

LEDGER ENTRY ANATOMY

| Field | Type | Purpose |
|---|---|---|
| `entry_id` | `uuid` | UUIDv7 — sortable, globally unique, generated at the terminal |
| `seq_no` | `bigint` | Server-assigned monotonic sequence; the canonical ordering |
| `entry_type` | `enum` | sale, refund, void, stock_move, waste, grn, adjustment, shift_close |
| `outlet_id` | `uuid` | Trading location, for multi-outlet consolidation |
| `shift_id` | `uuid` | Binds the entry to a bounded trading window |
| `amount` | `numeric(12,4)` | Signed monetary value; zero for pure quantity events |
| `quantity` | `numeric(14,4)` | Signed stock quantity in the ingredient's base unit |
| `reference` | `jsonb` | Links to order, bill, batch or purchase order as applicable |
| `actor_id` | `uuid` | Authenticated user responsible for the event |
| `device_id` | `text` | Originating terminal, for offline reconciliation |
| `prev_hash` | `bytea` | SHA-256 of the preceding entry in this outlet's chain |
| `hash` | `bytea` | SHA-256 over the canonical serialisation of this row |

A nightly integrity job recomputes the chain from the last sealed checkpoint and reports any discontinuity. A break in the chain is treated as a security incident, not a data-quality issue, and pages the operations on-call rota.

## 5.2 Daily Sales Record and Z-Report

At shift close the system produces a Z-report — the fiscal summary of that trading window. The report is written to the ledger as a sealed entry, and the shift is thereafter read-only.

Z-REPORT CONTENT

| Section | Content |
|---|---|
| Identity block | Outlet, register, shift ID, operator, open and close timestamps, Z number |
| Gross sales | By menu category, by tax class, by service type (dine-in, takeaway, delivery) |
| Discounts & promotions | Total by promotion code with an authorising manager for each override |
| Voids & refunds | Count and value, each with reason code and approver |
| Tender breakdown | Cash, card by scheme, voucher, house account, with expected vs declared cash |
| Tax summary | Net, tax and gross per tax rate, prepared for the statutory return |
| Integrity block | Entry count, first and last hash, seal timestamp |

## 5.3 Inventory and Batch Traceability

Inventory is not maintained as a single running number per ingredient. It is maintained as a set of batches, each traceable to a goods-received note and therefore to a supplier, an invoice and a delivery date. This is what makes a recall possible in minutes rather than days.

- **Depletion on sale** — when an order is settled, recipe lines are exploded into stock movements against batches in FEFO order (first-expired, first-out).
- **Batch identity** — lot number, supplier, receipt date, expiry date, unit cost and remaining quantity.
- **Adjustments** — a physical count variance is recorded as an adjustment entry with a reason code, never as an edit to a prior movement.
- **Traceability query** — given a lot number, the system returns every dish served from it and every guest bill affected.

## 5.4 Procurement and Supplier Records

Purchasing follows a simple four-document chain. Each document is immutable once issued; corrections are issued as new documents that reference the original.

PROCUREMENT CHAIN

| Document | Created by | Effect on records |
|---|---|---|
| Requisition | Kitchen or store, based on par-level shortfall | None — internal planning artefact |
| Purchase order | Purchasing manager, approved by GM above threshold | Commitment recorded; no stock movement |
| Goods received note | Store receiver at the door, with batch and expiry capture | Stock increases; batch records created; supplier invoice matched |
| Supplier invoice | Finance, three-way matched against PO and GRN | Payable recorded; price variance flagged |

## 5.5 Waste Logging and Tax Compliance

Waste is a first-class record, not a note on a clipboard. Every discard is logged with a reason, an estimated value and an authorising role, because unexplained waste is the single most common source of food-cost leakage.

### WASTE REASON CODES

| Code | Description | Approval required |
|---|---|---|
| EXP | Past expiry or use-by date | Store supervisor |
| SPOIL | Spoilage detected during preparation | Head chef |
| PREP | Trim and preparation loss beyond the recipe yield allowance | Head chef |
| RETURN | Guest returned a dish | Floor manager |
| DROP | Accidental drop or breakage | Shift manager |
| STAFF | Staff meal, recorded for cost allocation | Shift manager |
| TEST | Quality check or training sample | Head chef |

For tax purposes the ledger retains, for every trading day: gross and net turnover by tax rate, the tax collected, voids and refunds with authorisation, and the sealed Z-report hash. Records are retained for the statutory minimum period configured per jurisdiction — seven years by default — and are exportable to a signed, timestamped archive on demand.

CHAPTER SIX

# Business Intelligence & Analytics

Analytics reads from the ledger and the operational tables via a read replica. It never writes to operational state, which means a heavy month-end report can never slow down a Friday-night service.

## 6.1 Metric Definitions

Metrics are defined once, centrally, and referenced by name everywhere. This eliminates the classic problem of two dashboards showing two different "net sales" figures.

CORE METRIC DEFINITIONS

| Metric | Definition | Typical target |
|---|---|---|
| Net sales | Gross sales − discounts − voids − refunds, excluding tax | Budget ±3% |
| Average cover | Net sales ÷ covers served in the period | Set per outlet |
| Theoretical food cost | Σ (quantity sold × recipe plate cost) | ≤ 31% of net sales |
| Actual food cost | Opening stock + purchases − closing stock − staff meals | Within 1.5% of theoretical |
| Food cost variance | (Actual − theoretical) ÷ net sales | Under 1.5% |
| Gross profit | Net sales − cost of goods sold | ≥ 68% |
| Table turnover | Mean minutes from seated to free | Outlet-specific band |
| Ticket time | Mean minutes from Confirmed to Ready | ≤ 14 minutes |
| Void rate | Voided value ÷ gross sales | Under 0.8% |
| Waste ratio | Waste value ÷ purchases in the period | Under 2.5% |

## 6.2 Dashboard Catalogue

Four dashboards cover the operating rhythm of the business: the live service view, the daily close, the weekly commercial review and the monthly financial pack.

DASHBOARDS BY AUDIENCE

| Dashboard | Audience | Principal widgets |
|---|---|---|
| Live Service | Floor manager | Open tables, overdue tickets, 86 list, covers seated, current spend per table |
| Daily Close | Duty manager | Z-report summary, tender variance, voids by operator, waste logged today |
| Commercial Review | Owner / GM | Sales mix, menu-engineering matrix, day-part performance, channel split, week-on-week trend |
| Financial Pack | Finance | Food cost variance, gross profit by category, supplier price movement, stock valuation, tax summary |

## 6.3 Food Cost Variance Analysis

Variance analysis compares what the kitchen *should* have consumed, based on what was sold and the recipe cards, against what the stock ledger says was *actually* consumed. A persistent positive variance points to over-portioning, unrecorded waste or theft.

WORKED VARIANCE — ONE TRADING WEEK

| Line | Amount | Interpretation |
|---|---|---|
| Net sales | 48,600.00 | Baseline for percentage calculations |
| Theoretical food cost | 14,806.00 | 30.5% of net sales — within target |
| Actual food cost | 15,552.00 | 32.0% of net sales — 1.5 points high |
| Recorded waste | 612.00 | Explains roughly 1.3 points of the gap |
| Unexplained variance | 134.00 | 0.28% — inside tolerance, flagged for observation |
| **Action** | Re-weigh the three highest-cost recipe lines; audit trim practice on the fish station | |

## 6.4 Shift Audits and Exception Reporting

Rather than reviewing every transaction, managers review exceptions. The system surfaces a ranked exception list at each shift close, ordered by financial impact.

- **Discount concentration** — a single operator applying discounts far above the peer median for the same shift type.
- **Void clustering** — repeated voids on the same item or table within a short window.
- **Open-bill aging** — tables left open beyond a configured duration without a manager note.
- **No-sale drawer opens** — drawer events with no corresponding transaction.
- **Offline backlog** — terminals that replayed an unusually large queue after reconnection.
- **Price override** — any manual price change with its authorising credential and reason.

## 6.5 Data Export Formats

All exports are generated from a consistent snapshot so that two exports taken at the same instant always agree. Every export file carries a manifest with the snapshot timestamp, row counts and the ledger hash range it covers.

### EXPORT FORMATS

| Format | Intended use | Notes |
|---|---|---|
| `CSV` | Import into accounting or BI tooling | UTF-8, RFC 4180 quoting, one row per ledger entry |
| `XLSX` | Management review and modelling | Pre-formatted sheets with pivot-ready columns |
| `PDF` | Statutory filing and signature | Typeset Z-report with integrity block and page numbering |
| `JSON` | Integration with external systems | Versioned schema with a documented compatibility policy |
| `SQL dump` | Long-term archival and legal discovery | Compressed, checksummed, encrypted at rest |

CHAPTER SEVEN

# Security, RBAC & Disaster Recovery

A restaurant system holds commercially sensitive pricing, staff identity data and a complete financial record. It is treated as a regulated system, not a convenience tool.

## 7.1 Identity and Access Management

Every human actor has a named account. Shared logins are prohibited. Authentication uses a password hashed with Argon2id and, for any role with financial authority, a mandatory second factor.

- **Session model** — short-lived access tokens (15 minutes) with rotating refresh tokens bound to a device fingerprint.
- **Second factor** — TOTP for managers, administrators and finance; optional for service staff.
- **Terminal binding** — a POS session is tied to a named device; a session presented from an unknown device is rejected outright.
- **Elevation** — sensitive actions (voiding a sent item, reopening a bill, changing a price) require a fresh credential prompt, not just a live session.
- **Deprovisioning** — disabling an account revokes all tokens within 60 seconds and is itself an audited event.

## 7.2 Role Permission Matrix

Roles are additive but capped: no role may exceed the permissions of the role above it. The matrix below is the authoritative mapping and is enforced server-side on every request.

PERMISSION MATRIX (● GRANTED · — WITHHELD)

| Capability | Server | Cashier | Floor Mgr | Head Chef | Finance | Admin |
|---|---|---|---|---|---|---|
| Take and send orders | ● | ● | ● | — | — | ● |
| Void an unsent item | ● | ● | ● | — | — | ● |
| Void a sent item | — | — | ● | ● | — | ● |
| Apply discount up to 10% | — | — | ● | — | — | ● |
| Apply discount above 10% | — | — | — | — | ● | ● |
| Reopen a settled bill | — | — | ● | — | ● | ● |
| Edit menu items and prices | — | — | — | ● | — | ● |
| Edit recipes and yields | — | — | — | ● | — | ● |
| Record waste and adjustments | ● | — | ● | ● | — | ● |
| Create purchase orders | — | — | ● | ● | ● | ● |
| Receive goods and batches | ● | — | ● | ● | — | ● |
| Close and seal a shift | — | ● | ● | — | — | ● |
| View financial dashboards | — | — | ● | — | ● | ● |
| Export ledger and audit data | — | — | — | — | ● | ● |
| Manage users and roles | — | — | — | — | — | ● |
| View audit trail | — | — | ● | — | ● | ● |

## 7.3 Encryption and Key Management

ENCRYPTION POSTURE

| Data state | Mechanism | Key custody |
|---|---|---|
| In transit (external) | TLS 1.3, HSTS enforced, modern cipher suites only | Certificate from a public CA, auto-renewed |
| In transit (internal) | mTLS between service containers | Internal CA, 90-day leaf certificates |
| At rest — database | Full-volume encryption plus column-level encryption for identity fields | Key held in the host keystore, rotated annually |
| At rest — backups | AES-256 envelope encryption before leaving the host | Separate offline key share, split among two custodians |
| At rest — objects | Server-side encryption with object lock for WORM records | MinIO KMS, retention enforced by policy |
| Secrets | Injected at runtime from an encrypted store; never in images or repositories | Rotation on a 90-day schedule |

## 7.4 Backup, Recovery and Continuity

Backups follow the 3-2-1 rule: three copies, on two media types, with one copy off-host. Restoration is rehearsed quarterly, and an unrehearsed backup is treated as no backup at all.

RECOVERY OBJECTIVES

| Tier | Frequency | Retention | RPO | RTO |
|---|---|---|---|---|
| WAL archive | Continuous | 14 days | ≤ 5 min | 30 minutes |
| Nightly full dump | Daily 03:00 | 90 days | 24 hours | 60 minutes |
| Weekly cold copy | Weekly | 12 months | 7 days | 4 hours |
| Statutory archive | Monthly | 7 years | 30 days | 1 business day |
| Terminal local queue | Continuous | Until acknowledged | 0 | Immediate |

RPO — recovery point objective, the maximum acceptable data loss. RTO — recovery time objective, the maximum acceptable downtime before service is restored.

### Continuity in the outlet

If the server host fails entirely, terminals continue to trade from their local queues. A designated manager terminal can be promoted to a temporary peer authority to authorise voids and discounts, so the restaurant is not held hostage by a hardware failure. On restoration, the queues replay and the ledger reconciles with a full exception log of everything that happened during the outage.

## 7.5 Compliance and Retention

- **Financial records** — retained for a minimum of seven years, exportable in a non-proprietary format with integrity verification.
- **Personal data** — minimised at capture; staff identity fields encrypted at column level and access logged on every read.
- **Card data** — never stored; only authorisation references and masked digits are retained.
- **Audit trail** — retained independently of operational data and protected by object lock so it cannot be edited or deleted within the retention window.
- **Right of access** — a data subject request can be satisfied by a scoped export that includes every record referencing the individual.

> ACCEPTANCE GATE
>
> No production cutover is permitted until a restore rehearsal has been completed from the WAL archive within the stated RTO, and the integrity job has verified the full hash chain from the first sealed checkpoint to the present day.

END OF DOCUMENT · LNX-SPEC-2024-001 · V1.0